

Smart Cycle Helmet: An Embedded System for Enhanced Rider Safety

Mazen Jrab (Team Leader)
Purdue University
mjrab@purdue.edu
Frankfurt, Germany

Aathira Hegde
Purdue University
hegde45@purdue.edu
Reading, UK

Colin Chambers
Purdue University
cjchambe@purdue.edu
West Lafayette, Indiana

Justin Hasenstab
Purdue University
JHasenst@purdue.edu
Boston, Massachusetts

Aya Elghayaty
Purdue University
aelghaya@purdue.edu
Cairo, Egypt

Kareem Hassan
Purdue University
hassa112@purdue.edu
Cairo, Egypt

Matthew Roche
Purdue University
roche22@purdue.edu
Stamford, Connecticut

Tyler Weltzin
Purdue University
Tweltzin@purdue.edu
Eau Claire, Wisconsin

Abstract - Cycling safety technology has seen significant advancements, but existing solutions suffer from high power consumption, limited accuracy, unstable Bluetooth connections, and rudimentary detection methods. This project aims to develop an embedded system for a smart bicycle helmet with advanced safety features, including crash detection, fall detection, and automatic braking detection. The system will integrate IMUs and capacitive sensors while employing machine learning algorithms to improve event classification accuracy. Bluetooth Low Energy (BLE) will enable seamless communication with existing mobile apps to send SOS alerts and visualize collected data. By addressing current market limitations, this project will enhance cycling safety, ensuring better rider protection and awareness

Keywords—Crash Detection, Machine Learning, Cycling Safety, BLE, IMU.

I. INTRODUCTION

With the increasing popularity of cycling, there is a growing demand for improved safety solutions. Existing bicycle helmet systems often fall short due to various issues and limitations that compromise the overall reliability. This project aims to design an embedded system for a smart bicycle helmet that enhances safety through advanced crash detection, fall detection, and automatic braking detection. The system will integrate multiple sensors and employ machine learning (ML) techniques to improve accuracy and reliability.

II. MOTIVATION AND OBJECTIVES

Cycling safety is a growing concern, yet existing crash and fall detection systems have major drawbacks such as high-power consumption, limited accuracy, unstable Bluetooth connectivity, and basic detection mechanisms. We are motivated to improve safety for cyclists and encourage people to ride more bicycles and feel safer by addressing these drawbacks.

The primary goals of this project are:

- Develop an embedded system for bicycle helmets that supports crash detection using Inertial Measurement Units (IMUs).
- Enable Bluetooth Low Energy (BLE) communication with a companion mobile app to send SOS alerts, including GPS location, to emergency contacts in the event of an accident.
- Implement fall detection to identify when a rider has fallen and trigger an appropriate safety response.

- Integrate automatic braking detection, activating a rear light on the helmet when the user applies the brakes.
- Address existing market drawbacks by improving power efficiency, accuracy, and connectivity stability.

III. STATE OF THE ART RESEARCH

Current research investigates multiple approaches to crash detection. Camera-based systems utilize machine learning to identify car accidents, while in-car sensors leverage multimodal data for automated detection. A survey has been conducted to analyze real-time accident detection trends using AI. Additionally, vital sign monitoring is being explored with deep learning models to enhance safety. IMU-based systems have also been developed for crash detection, and our project builds on this foundation, aiming to create a more efficient and scalable IMU-based solution tailored for cyclists [1]–[7].

IV. POTENTIAL APPLICATIONS

Our crash detection system has a wide range of applications beyond cycling safety. In addition to helmet integration, it can be embedded into bike frames and wearables for real-time safety monitoring. Beyond crash detection, the system can be further enhanced to detect rider fatigue by analyzing head movements. Additional sensors could be integrated to monitor body temperature and vital signs, expanding its functionality for health and safety applications.

The technology has the potential to scale beyond cycling, extending to motorcycles and elderly wellness devices. Furthermore, it can be integrated with V2X systems to alert nearby vehicles in the event of an accident, enhancing road safety.

To ensure emergency alerts even in areas without cellular coverage, the system could be developed to utilize Non-Terrestrial Networks (NTN), enabling timely rescue operations in remote locations.

V. SYSTEM OVERVIEW AND ARCHITECTURE

The system leverages embedded hardware and machine learning to identify critical events such as crashes, falls, and braking. By enabling timely alerts and visual feedback, the system serves both as a proactive safety tool and an emergency response aid.

At the core of the system is an Inertial Measurement Unit (IMU) that continuously monitors motion and orientation data. This sensor data is processed locally on the

microcontroller using a trained machine learning model capable of recognizing different motion patterns associated with crash, fall, braking, and normal riding events. The model runs in real time, allowing the system to react immediately when such events are detected.

In the event of a crash or fall, the system alerts over a Bluetooth Low Energy (BLE) connection a companion mobile application. The app, upon receiving the alert, automatically sends an SOS message along with the rider's GPS location to pre-configured emergency contacts. This ensures rapid intervention and improves the likelihood of receiving help in time.

To further enhance visibility and rider safety, the system includes an integrated rear light. This light is triggered automatically during braking events, alerting nearby vehicles and cyclists to the rider's deceleration, thereby helping to prevent rear-end collisions.

To improve detection accuracy and reduce false positives particularly while the bike is stationary or being moved without a rider, we have integrated a capacitive saddle sensor that connects over BLE to the App as well. This sensor determines whether the cyclist is seated, adding valuable context to motion data and refining the decision-making process of the event detection system.



Fig 1. System Architecture

VI. HARDWARE DESIGN

The system consists of the following components:

- **ESP32-Pico-Mini-02:** We selected the ESP32-PICO-MINI-02 as the main processing unit for its compact size, integrated Bluetooth capabilities, and support for deep sleep modes which is crucial for optimizing power usage. It provides sufficient computational resources to run a lightweight machine learning inference engine using Micropython, handle BLE communication with the mobile app, manage real-time sensor data processing, and control peripheral components such as the rear light.
- **IMU LSM6DSOX:** Being an integral part of the application, the 6-axis IMU, 3-axis accelerometer to sense linear acceleration and 3-axis gyroscope to sense angular acceleration, in a single package combining high performance with low power consumption. The device supports serial links such as I2C and SPI for communication.

The LSM6DSOX was chosen over other IMUs due to its unique combination of high precision, low power consumption, and advanced embedded features such as machine learning capabilities, dynamic data batching, and activity detection. These features reduce the need for extensive external processing, optimize power

efficiency, and enable seamless integration into resource-constrained embedded systems, making it superior to alternatives that lack such built-in functionality.

The IMU supports dynamic data batching capability for the sensors. The data is stored in a FIFO buffer and sent as a burst of data to the ESP32 when the FIFO is full. This helps reduce the communication overhead for the ESP32 and helps store data for processing. The inactivity/activity detection feature is used to detect if the rider is in stationary or moving with the helmet on. If the system remains inactive for a while, the IMU goes into a low-power mode and notifies the ESP32 to go into deep sleep state. When the IMU detects a movement, it wakes the ESP32 up to start running, collecting and processing data, and detecting any event. Both the features play a critical role in reducing the power consumption of both the IMU and the ESP32.

- **Capacitive Sensor on Saddle:** we integrated a capacitive touch/proximity sensor (SEN-12041) beneath the bicycle saddle. This sensor helps the system determine whether the cyclist is seated. Its primary function is to reduce false positives from crash or fall detection.

The SEN-12041 provides a simple digital output based on capacitive field changes, which we interpret as either “seated” or “not seated.” When the sensor detects that the rider is not present the embedded system adjusts the behavior of the crash/fall detection algorithm by applying a higher confidence threshold before classifying an event as a crash or fall.

This small but critical component adds robustness to our real-world deployment, particularly during idle or transportation scenarios.

- **Rear Brake Light:** For rear light control, a high-brightness LED module is mounted on the back of the helmet. It is activated through a general-purpose I/O (GPIO) pin on the ESP32, controlled via software logic in response to braking events:
 - When braking is detected by the ML model, the ESP32 drives the GPIO pin high for 3 seconds, turning the light on.
 - After the timer expires, the GPIO is pulled low, turning the light off.

This braking indication mechanism serves a similar role to brake lights on vehicles, improving visibility particularly in low light conditions and alerting road users that the cyclist is slowing down.

VII. SENSOR INTEGRATION

Two-wire I2C interface is used for communication between ESP32 and LSM6DSOX IMU with the processor in master mode. A maximum of two bytes of data transfer in one communication cycle is expected for the application. Prior to loading the configuration, the application verifies the communication link by reading the contents of the WHO_AM_I_REG register in the device. If the read contents match the expected value (0x6C), the system loads the configuration settings onto the IMU.

FIFO Configuration: For the accelerometer, the output data rate is set to 26Hz and full-scale range to $\pm 16g$ using CTRL1_XL register. For the gyroscope, the output data rate is set to 26Hz and full-scale range to $\pm 2000dps$ in the CTRL_G register.

Every data word in the FIFO consists of one byte of FIFO_DATA_OUT_TAG (indicates the type of sensor data) and 6 bytes of the corresponding data (2 bytes of X, Y and Z). The FIFO threshold is set to 1 second of accelerometer and gyroscope data, 52 words (0x34) in total, in the FIFO_CTRL1 register. The FIFO depth is limited to the threshold by setting the MSB of FIFO_CTRL2 register high. The batching rate of the gyroscope and the accelerometer is set to 26Hz by configuring FIFO_CTRL3 and the FIFO is set to continuous mode in FIFO_CTRL4. INT1 is configured to go high whenever the FIFO threshold limit is reached. When the FIFO threshold is reached ($\sim 1s$), the processor is interrupted and reads the FIFO status register for the number of unread samples in the FIFO. The processor then reads all the unread samples and stores them as a list. This data is passed onto the ML core in the main application or transferred to the PC via Bluetooth for data collection.

Inactivity detection: The inactivity/activity feature is used to detect if the cyclist is stationary or in motion. When stationary, the system goes into sleep mode reducing system power consumption. The feature is activated by enabling the feature in TAP_CFG2. When inactivity is detected, the accelerometer goes to low-power mode (output data rate - 12.5Hz) and the gyroscope in power-down mode. The wake-up duration threshold is set at 115ms while the sleep duration is set to 19.69s in the WAKE_UP_DUR register. The wake-up threshold is set at 0.75g in the WAKE_UP_THS. This means, if it sees the acceleration value on any axis go beyond 0.75g for a duration of 115ms, the IMU goes into active state. If the acceleration remains below the threshold for 19.69s, it goes to low-power mode. The sleep status is reported to INT2 pin. Any edge on INT2 interrupts the processor – if the level is low, the processor goes into a deep sleep mode and if the level is high, the processor wakes up and starts collecting and processing the data. This feature is enabled only in the main application.

VIII. BLE COMMUNICATION IMPLEMENTATION

The two ESP32s were implemented as BLE Generic Attribute Servers (GATT servers) which allowed each individual device to house a specific Service and Characteristic that would be used for reading/writing data. After initializing, the ESP32 devices begin advertising themselves as generally discoverable devices with their specific device name encoded and append to the end of the advertisement package for clients to connect to.

Once a connection request is received from either the users' phone or PC and accepted by the ESP32, the parties involved agree to a Maximum Transmission Unit (MTU) which determines the maximum number of bytes that are available to be sent over in one transfer. Once all connection handshakes are successful, the ESP32s begin monitoring their specific instruments for critical events. If a critical event

is detected, the ESP32 will write the updated byte data to the characteristic field and notify the connected device that an update has been made. In turn the connected device will receive this notification, read the new byte values, and act on them appropriately.

Upon disconnection, the ESP32 devices will stop monitoring their instruments and start advertising again looking for a new connection. The BLE initialization and communication implementation flow chart is illustrated in the following figure.

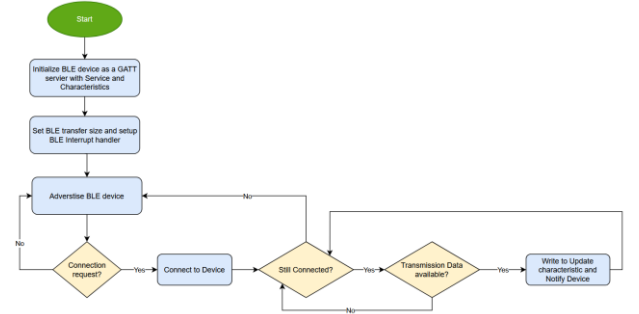


Fig 2. BLE Communication Flow Chart

IX. DATA COLLECTION

For data collection, we added an external push button to the overall design. When this push button was triggered, it would open a five second event recording window. In this window, we collected the IMU data for a specific critical event that we wanted to be able to detect.

We developed a modified Windows PC Python script that would utilize the Bleak library [8] to allow for BLE communications with the ESP32. With this script, we were able to successfully connect to the ESP32 over BLE and establish a 512-byte MTU allowing us to receive all the data collected in the event recording window. After the byte data was transferred to the Windows PC, we decrypted the data, added an index to track different event recording windows, and finally appended the data into a .csv file which was used for the machine learning training. The source code for this script is available on [the project's GitHub repository](#).

Using this method, we were successfully able to record 300 normal riding events, 100 crash events, 100 braking events, and 100 fall events.

X. MACHINE LEARNING

We implemented a multiclass classification machine learning approach to analyze data collected from the IMU. The goal is to accurately classify events into one of four categories: crash, fall, braking, or normal riding. By leveraging sensor data such as acceleration and angular velocity, the model is trained to recognize patterns associated with each event type. The approach followed, including data preprocessing, feature engineering, and model architecture, and model evaluation is detailed in the following subsections.

A. Data Preprocessing

The raw sensor data from the 6-axis IMU (accelerometer and gyroscope) was collected at 26Hz, resulting in a large

volume of time-series data. We designed custom detection logic to isolate the most informative 1-second window for each event:

The continuous sensor data was divided into fixed-length windows to capture meaningful motion patterns. Each window was labeled based on the event it represented: (crash, fall, braking, or normal riding).

- Crash → Second with maximum jerk peak.
- Braking → Second with high jerk_x_mean and negative accel_x_mean.
- Fall → Second with maximum jerk peak.
- Normal → Random 1-second windows from continuous riding.

B. Features Engineering

To extract meaningful information from the raw sensor data, we computed the following features for each window, where 13 kinematic features were extracted, from each 1-second window:

- Acceleration/Gyroscope Means: Axis-wise averages (x, y, z).
- Jerk (Rate of Change): Critical for detecting sudden movements (e.g., crashes).
- Peak Magnitudes: Including accel_z_peak (vertical impact).

These features were selected to capture the distinct motion signatures associated with crashes, falls, and braking events.

Final Dataset:

- 100 samples per class (Crash, Fall, Braking).
- 300 samples for Normal riding.

C. Model Architecture

We evaluated several machine learning algorithms for event classification, including Random Forest, Support Vector Machines (SVM), and k-Nearest Neighbors (k-NN). The **Random Forest classifier** was chosen for its robustness, interpretability, and ability to handle high-dimensional feature spaces.

- **Hyperparameters:**
 - Number of estimators (trees): 10
 - Maximum depth: 5
- **Training:** The model was trained on 80% of the dataset (394 samples).
- **Inference:** The trained model was ported to the ESP32 microcontroller, enabling real-time classification on the edge.

D. Model Evaluation

The model's performance was evaluated on a held-out test set (20% of the data, 99 samples) using the following metrics:

1. **Test Accuracy:** 91%. To ensure the model's robustness and generalization, we also performed 5-fold cross-validation, which yielded a mean accuracy of 95.5% with a standard deviation of $\pm 3.0\%$.
2. **Precision, Recall, and F1-Score:** The model's performance was evaluated using precision, recall,

and F1-score, with crash detection prioritized for safety—achieving a perfect 100% recall (no missed crashes) and 95.7% precision (minimal false alarms). Other classes like falling and braking/normal maintained strong reliability. The high recall for crashes ensures life-saving alerts, while slightly lower precision tolerably balances false warnings.

	Precision	Recall	F1-score
Normal	87.5%	90.3%	89.0%
Crash	95.7%	100%	98.0%
Braking	88.9%	88.9%	89.0%
Falling	94.1%	84.2%	89.0%

Table 1. Precision, Recall and F1-Score

The confusion matrix (by row) is also shown below. It demonstrates the model's strong performance in classifying the events especially crash events achieving no false negatives and 100% recall as it is our most prioritized event and achieving great accuracy scores on the other events as well.

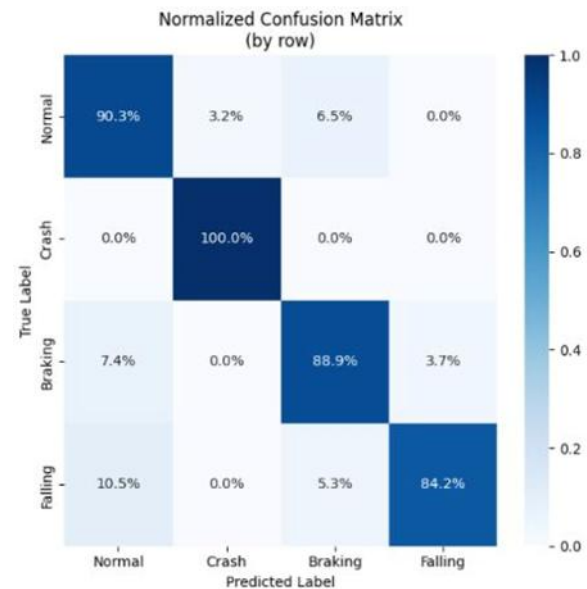


Fig 3. Confusion Matrix

The bar chart below illustrates the classification performance of a detection system across four activity categories: Normal, Crash, Braking, and Falling. For each class, it displays the number of True Positives (green), False Positives (orange), and False Negatives (red).

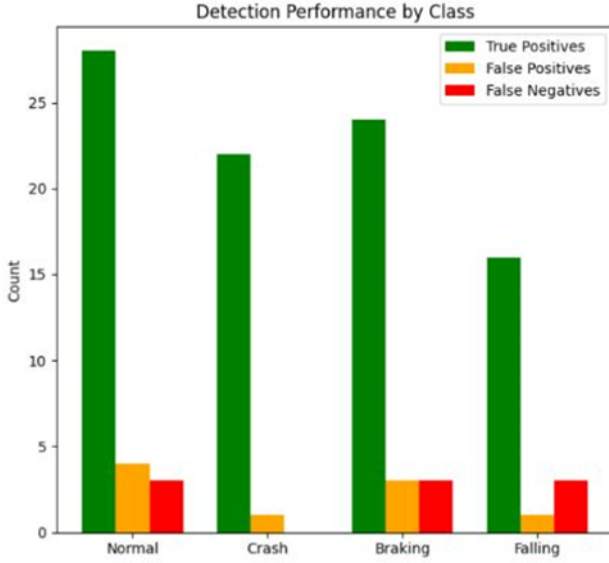


Fig 4. Detection Performance by Class

XI. MAIN APPLICATION DEVELOPMENT

Building upon the components and methods described in the previous sections, the main application has been developed to integrate data acquisition, event classification, and system response in a unified, real-time process. The application runs on the helmet's embedded system and continuously collects data from the IMU, processes it, and applies the trained machine learning model to classify each event as a crash, fall, braking, or normal riding.

The application follows a structured and modular design to ensure robustness, low latency, and scalability. It incorporates sensor calibration, preprocessing, feature extraction, and classification within a loop that operates by interrupts. System responses such as alerting, logging, or communication with the mobile App or rear light are triggered based on the classification result.

To conserve power, the application enters deep sleep mode after ~20 seconds of inactivity to reduce energy consumption. The IMU remains active in a low-power mode and is configured to trigger an interrupt when motion is detected. This interrupt wakes the system up, which then resumes normal operation starting from IMU FIFO configuration. This mechanism ensures efficient power usage while maintaining responsiveness to new events. The complete source code for this project is available on [the project's GitHub repository](#).

The flow of the main application is illustrated in the following state machine diagram.

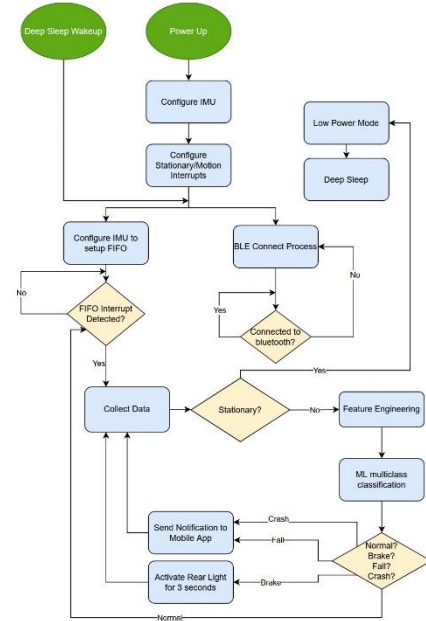


Fig 5. Main Application Flow Chart

XII. APP DEVELOPMENT

The separate web application was developed using the Flutter framework which was chosen for its ability to deploy cross-platform apps from a single Dart codebase. This allowed us to test and deploy the app easily on both mobile devices and the web. Flutter's widgets features allowed us to quickly create a responsive dashboard UI which would have been significantly more time-consuming using vanilla HTML and CSS.

The app interacts with the helmet's onboard sensors via the flutter_web_bluetooth package, which allows real-time Bluetooth Low Energy (BLE) communication within supported web browsers. Users can connect to the helmet, monitor live acceleration data, and receive crash alerts. When the firmware on the ESP32 detects a crash, it sends a BLE notification to the app. The app subscribes to these notifications, allowing it to react instantly by displaying a 30-second countdown timer. If the user does not cancel within this time, the app automatically initiates a call to a designated emergency contact. Additionally, the app sends an SOS message containing the user's current GPS coordinates, providing first responders with critical location information. In future versions, we hope to make the countdown duration user-configurable. Additionally, a student during the presentation recommended an audio alert to go with this to properly alert the user that a crash has been detected, which would be ideal for a future release of the app.

One of the key challenges during development was dealing with the limitations of Web Bluetooth APIs, especially the inconsistent support across browsers and platforms. For example, many devices appeared as "unknown" in the selection UI due to missing metadata in ESP32 BLE advertising packets. To work around this, device identification was performed through manual UUID

matching and testing. Additionally, debugging BLE behavior within the web environment required careful synchronization between the firmware and app layers. Despite these hurdles, our final result is a user-friendly and safety-oriented web app that works on multiple browsers and device types.

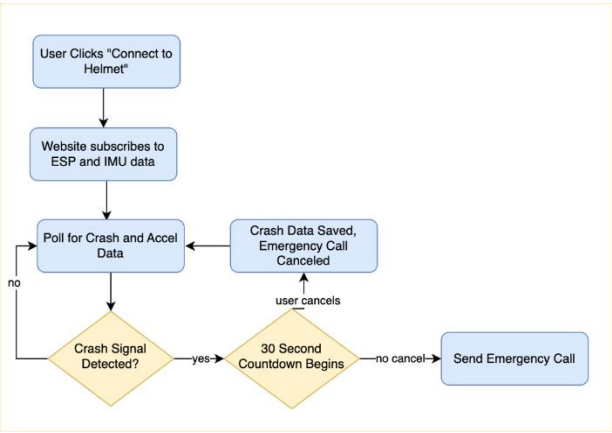


Fig 6. Web App Flow Chart

XIII.ELECTRICAL AND MECHANICAL DESIGN

To get to a more realized state, the circuitry was fully customized as opposed to using the ESP32 Feather board. The circuit schematic is seen in figure (7). Included in this schematic is the ESP32, LSM6DSOX IMU, and battery management circuit. The battery management allows charging via USB-C for a 3.7 V LiPo battery.

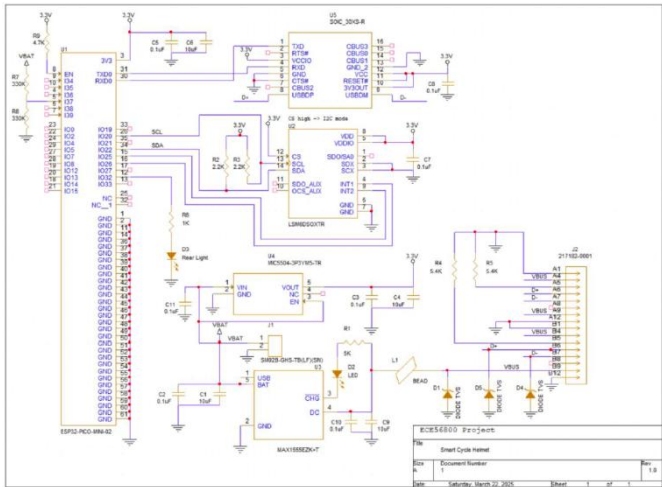


Fig 7. Circuit Schematics

The custom circuit allowed for a PCB to be designed with a minimal footprint. The PCB in figure (8) measures 46.8 x 18.0mm.

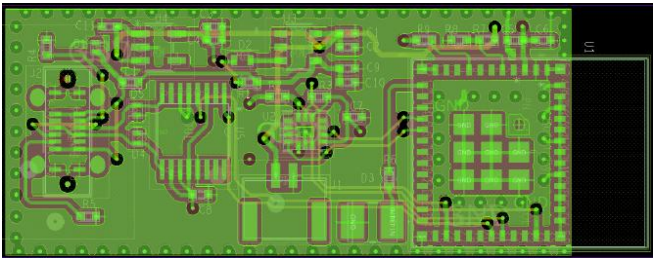


Fig 8. Circuit Layout

In addition to the electrical design, the mechanical components and housing were designed. Figure (9) shows a slice of the design with the brake light housing (1), USB-C charging cover (2), PCB (3), LiPo battery (4), connecting clip (5), and helmet attachment (6). The helmet attachment accounts for different helmet designs by allowing a strap to pass through that can be attached to helmets with holes or gaps. Alternatively, the attachment will come with a sticker like paste to semi-permanently attach to a smooth helmet. The attachment measures 87.2 x 45.5 x 44.9 mm creating minimal impact for the biker.

A primary goal for the housing was for it to be water resistant. Three steps were taken to achieve this. First, a thin rubber seal was placed between the red plastic cover and the rest of the housing. Second, the outer stews went directly into the top level. This is opposed to having a bolt on the inside, which would have a greater risk of water leaking in. Lastly, a multi-step cover with an additional rubber seal was designed for the USB-C Charging port.

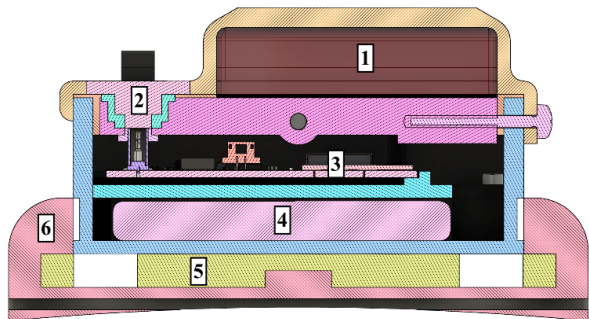


Fig 9. Slice of Mechanical Design

Figure (9) shows a rendered view of the design. In the rendered view, the clipping mechanism can more easily be seen. To use it, the user simply presses in the two black tabs to release the housing from the white helmet attachment.



Fig 10. Rendered view of mechanical design

Work that still needs to be done for the housing and attachment includes specific material selection and how it will be manufactured. Both may have impacts on details of the design, specifically on the thickness and/or the size of each component. However, this initial design provides a strong foundation to continue to that next step.

XIV.EVALUATION

Our system was evaluated based on functionality, performance, power efficiency, and reliability. Each core component was tested individually and as part of the

integrated system to ensure end-to-end stability and effectiveness in real-world cycling scenarios.

A. Experimental Setup

The final prototype was assembled on a bicycle helmet and tested under controlled indoor conditions. Key metrics such as detection accuracy, power consumption, Bluetooth stability, and firmware execution times were recorded. The system was powered using an 800 mAh LiPo battery during field testing.

B. Hardware Integration:

The physical design successfully integrated the ESP32-PICO-MINI-02, LSM6DSOX IMU, capacitive saddle sensor, and rear LED light into a robust, lightweight, and ergonomic helmet form factor. This ensured minimal interference with rider comfort or helmet usability.

C. Event Detection with Machine Learning:

The machine learning model achieved 91% classification accuracy across four classes: crash, fall, braking, and normal riding. This was validated using labeled test data collected from cycling events. Feature extraction and model inference were fully embedded on the ESP32 and performed within tight real-time constraints:

- Feature engineering execution time: ~2 ms
- ML model inference time: ~410 μ s

D. Bluetooth Connectivity:

The BLE connection between the helmet and the mobile application remained stable during all tests, with reliable alert delivery and data synchronization. Alert messages triggered by crash/fall events were successfully received and handled by the app with a 100% delivery success rate in our final trials.

E. Firmware Performance:

The firmware efficiently managed sensor communication, data buffering, and ML inference. Key performance highlights include:

- FIFO buffer read time from the IMU: ~48 ms
- Total cycle time for data processing and classification: under 55 ms

F. Power Consumption:

Power optimization strategies—using deep sleep modes, activity/inactivity detection, and event-driven interrupts—resulted in excellent energy efficiency:

- Sleep mode current draw: ~0.05 mA
- Active mode current draw: ~47 mA

Estimated battery life: ~17 hours of continuous active riding on a single 800 mAh LiPo charge

G. Mobile Application:

The companion app was fully implemented to handle BLE alerts and send SOS messages with GPS coordinates. In testing, all crash and fall alerts triggered a countdown, followed by successful emergency call if unacknowledged.

XV. TEAM COMPOSITION AND JUSTIFICATION

Our team consists of eight members, exceeding the usual maximum of seven, due to our diverse and geographically distributed composition. We have team members from Egypt (2), the UK (1), the US (4), and Germany (1). While this international collaboration brings valuable perspectives and expertise, it also introduces challenges in efficiency due to time zone differences.

To justify our larger team size, we have expanded the scope of our deliverables. Beyond software and embedded system development, we've committed to providing a more comprehensive and robust solution by designing a custom PCB, rather than relying solely on development boards as originally planned. Additionally, we took the project a step further by designing a mechanical attachment for the helmet, improving the integration and practicality of our crash detection system. Moreover, A mobile App was developed for better user interaction. This expanded scope ensured that we've fully leveraged our team's expertise to achieve these ambitious goals efficiently by working in smaller squads as the roles table shows.

XVI. LIMITATIONS

While our system demonstrated strong performance in controlled testing scenarios, there are several limitations that must be acknowledged.

A key constraint in our project was the inability to collect real-world crash data due to safety and ethical concerns. As a result, our machine learning model was trained and validated using simulated or controlled crash and fall events. Although these simulations were carefully designed to resemble realistic cycling incidents, they may not fully capture the variability and unpredictability of actual crashes, especially those involving external factors like collisions with vehicles, uneven terrain, or complex rider behavior.

Another limitation lies in the size and diversity of our dataset. With more data specifically from a wider range of riders, riding styles, and environmental conditions, the model could be better generalized and less prone to false positives or missed detections. Currently, the system performs well within the tested conditions, but its behavior under less-represented edge cases (e.g., riding on gravel, jumping curbs, or abrupt dismounts) is less certain.

To fully validate the system's effectiveness, long-term field testing with volunteer riders and potentially anonymized community-collected data would be necessary. Incorporating additional sensors, like GPS or barometric pressure sensors, may also help improve context awareness and detection precision.

Overall, while our results are promising, we believe that collecting more real-world data and expanding the training set are crucial next steps to further enhance the reliability and robustness of the system.

XVII. FUTURE WORK

To improve system performance and robustness, several future directions can be explored:

A. Real-World Data Collection:

Establishing a protocol for safe, anonymized collection of real crash and ride data from volunteer cyclists would

significantly enhance model accuracy and reduce the gap between controlled testing and real deployment conditions.

B. Dataset Expansion and Diversification:

Collecting data from a larger pool of users across different demographics, bicycles, environments, and riding styles would enable the model to generalize better and reduce bias.

C. Sensor Fusion:

Integrating additional sensors such as barometric pressure sensors, or even microphones could provide richer context for event classification and reduce ambiguity in complex scenarios.

D. Online Learning or Adaptive Models:

Implementing a system that can continuously improve over time, either through retraining or user feedback, would make the device smarter and more responsive in the long run.

E. User Feedback Loop:

Adding a mechanism for users to confirm or reject detected events via the app can help improve model labeling and refine detection logic.

By addressing these limitations, we aim to move closer to a production-ready system that can be safely and confidently deployed on a wide scale for real-world cyclist protection.

XVIII. CONCLUSION

This project successfully delivered a reliable and intelligent helmet safety system that enhances cycling safety through real-time event detection and emergency response capabilities. By integrating a 6-axis IMU, a capacitive saddle sensor, Bluetooth communication, and an embedded machine learning model, we achieved high-accuracy detection of crashes, falls, and braking events. Our system improves upon existing solutions by offering better detection accuracy, robust power management, and seamless connectivity with a companion mobile app. The result is a practical, energy-efficient, and user-friendly safety enhancement for cyclists, contributing meaningfully to rider protection and situational awareness.

Team Member Responsibilities and Roles

Project Proposal	HW Selection	Sensor Integration	Bluetooth Connection	Data Collection	Machine Learning	Hardware Design	App Visualization
Mazen	Mazen	Mazen	Justin	Mazen	Aya	Mazen	Colin
Aathira	Aathira	Aathira	Colin	Aya	Kareem	Matthew	Justin
Justin	Tyler	Justin		Kareem	Mazen	Tyler	
Kareem	Matthew	Tyler		Justin			
Colin	Aya	Matthew		Aathira			

Table 2. Team Contribution

References

- [1] M. Arif, A. Rahaman and A. K. M. M. Islam, "Car Crash Detection System using Machine Learning and Deep Learning Algorithm," *ResearchGate*, Oct. 2022. [Online]. Available: https://www.researchgate.net/publication/364450064_Car_Crash_Detection_System_using_Machine_Learning_and_Deep_Learning_Algorithm
- [2] M. Alsharif, M. M. A. Elbamby, M. Alosaimi and A. A. Alturki, "A Machine Learning Framework for Automated Accident Detection Based on Multimodal Sensors in Cars," *Sensors*, vol. 22, no. 9, p. 3482, May 2022. [Online]. Available: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC9146681/>
- [3] V. Joshi and A. Raut, "Real-Time Accident Crash Detection using Machine Learning: A Survey," *ResearchGate*, Feb. 2024. [Online]. Available: https://www.researchgate.net/publication/377686339_Real-Time_Accident_Crash_Detection_using_Machine_Learning_A_Survey
- [4] M. Santhosh, V. Mohanraj and K. Dinesh, "Accident Detection System," in *Proc. 2020 4th International Conference on Trends in Electronics and Informatics (ICOEI)*, Tirunelveli, India, 2020, pp. 221–224. doi: 10.1109/ICOEI48184.2020.9186156.
- [5] M. D. K. Sudharsan, K. R. Deepika, R. S. M. Priya and P. S. Kumar, "Accident Detection and Alerting System using GPS and GSM," in *Proc. 2017 IEEE International Conference on Intelligent Techniques in Control, Optimization and Signal Processing (INCOS)*, Srivilliputhur, India, 2017, pp. 1–4. doi: 10.1109/ITCOSP.2017.7968631.
- [6] R. Kumar and S. Iqbal, "Intelligent Accident Detection and Alert System for Emergency Medical Assistance," *International Journal of Scientific and Technology Research*, vol. 5, no. 10, pp. 254–258, Oct. 2016. [Online]. Available: <https://sciresol.s3.us-east-2.amazonaws.com/IJST/Articles/2016/Issue-10/Article38.pdf>
- [7] M. I. Zulkarnain et al., "Development of a Cycling Safety Services System and its Deep Learning Bicycle Crash Model," in *Proc. 2022 IEEE International Conference on Consumer Electronics - Asia (ICCE-Asia)*, Kuala Lumpur, Malaysia, 2022, pp. 1–6. doi: 10.1109/ICCE-Asia56264.2022.9755093.
- [8] "Bleak - Bluetooth Low Energy platform Agnostic Klient," GitHub, [Online]. Available: <https://github.com/hbldh/bleak>. [Accessed: May 7, 2025].